A One-Shot Wakeup Boost for Long Uninterruptible Sleeps in the Linux Fair Scheduler

Francesco Territo David Carlino

August 2026

Abstract

The Linux fair scheduler is designed to distribute processor time between tasks while also preserving their responsiveness. Linux 6.13.9 uses the Earliest Eligible Virtual Deadline First (EEVDF) policy for this purpose. However, this fairness can disadvantage I/O-bound processes during high CPU contention, leading to increased latency. This report documents the implementation and benchmarking of a wakeup boost mechanism for Linux kernel 6.13.9. The final implementation applies a temporary, one-shot boost to tasks waking after a long uninterruptible sleep. Its performance must be measured again before publishing quantitative results.

1 Introduction and Motivation

I/O-bound processes often voluntarily yield the CPU to wait for resources. When they wake up, they rejoin the runqueue and can still spend time waiting behind other eligible tasks during high CPU contention, resulting in poor responsiveness. The objective of this project was to implement a specific optimization within the fair scheduler to favor these wakeups. The proposed solution introduces a **wakeup boost mechanism** that temporarily lowers the virtual runtime (`vruntime`) of tasks waking after a long uninterruptible sleep. It does not change their static priority. This modification aims to reduce runnable wait time, measured through the `wait_sum` scheduler statistic; thereby improving interactivity in mixed-workload scenarios.

2 Background (Fair scheduler internals)

1. Each CPU has its own struct rq (runqueue), which keeps track of runnable processes on the CPU. The struct cfs_rq is a sub-structure that represents the CFS-specific runqueue (there can be one per CPU and per scheduling group).

2. Each task has a vruntime (Virtual Runtime), which represents weighted execution time and grows according to the task's scheduling weight. If a task uses the CPU, its vruntime grows. In Linux 6.13.9, vruntime is also used to calculate lag, eligibility and the virtual deadline used by EEVDF.

3. Runnable scheduling entities are inserted into a RB (balanced) tree; with EEVDF this tree is ordered by virtual deadline rather than simply by vruntime. The scheduler chooses an eligible task with the earliest virtual deadline. Therefore, the selected task is not always just the one with the smallest vruntime, even though changing vruntime can influence both eligibility and deadline calculation. The runqueue also tracks min_vruntime as its progressing virtual-time reference. Time needs to search, delete or add a task is: $O(\log n)$, where n is the number of entries in the tree.

It is also useful to describe these functions, which will be used from now onwards:

- copy_process(): It is the main function for the creation of a new process or thread. It allocates and initializes the structure task_struct for the new task.

- enqueue_task_fair(): Used to insert a task in the runqueue (it is a structure that tracks all tasks that are ready to run on a CPU), updating its scheduling state so that EEVDF can evaluate eligibility and virtual deadline.
- place_entity(): Function that places a waking or newly created scheduling entity by updating its vruntime, lag and virtual deadline.
- dequeue_task_fair(): Handles a task leaving the fair runqueue due to sleep or termination, including EEVDF's delayed-dequeue case.
- try_to_wake_up(): Manages the awakening of a sleeping task, more in detail:
  1. Checks if the task is sleeping
  2. Sets the task status to "running"
  3. Chooses the CPU on which to enqueue the task. In the benchmark, QEMU exposes two virtual CPUs and the target workload is pinned inside the guest.

3 Implementation

3.1 Kernel Modifications

The modifications target the Linux 6.13.9 kernel source, specifically `include/linux/sched.h`, `kernel/sched/fair.c` and `kernel/fork.c`.

3.1.1 Data Structure Changes

The `task_struct` definition was extended to track the timestamp of the last uninterruptible sleep and a flag for boost eligibility:

```
struct task_struct {
  /* ... existing fields ... */
  /* One-shot wakeup boost after a long uninterruptible sleep. */
  u64 last_uninterruptible_sleep_start;
  unsigned wakeup_boost_pending:1;
  /* ... */
};
```

Listing 1: Changes to include/linux/sched.h

The wakeup-boost fields are initialized in `kernel/fork.c`:

```
  /* Do not inherit wakeup boost state from the parent. */
  p->last_uninterruptible_sleep_start = 0;
  p->wakeup_boost_pending = 0;
```

Listing 2: Changes to kernel/fork.c

The core scheduler modifications are located in `kernel/sched/fair.c`:

```
static void prepare_wakeup_boost(struct rq *rq,
          struct task_struct *p, int flags)
{
  s64 sleep_time;

  if (!(flags & ENQUEUE_WAKEUP))
    return;

  p->wakeup_boost_pending = 0;
  if (!p->last_uninterruptible_sleep_start)
    return;

  sleep_time = (s64)(rq_clock(rq) -
         p->last_uninterruptible_sleep_start);
  p->last_uninterruptible_sleep_start = 0;
```

```
16
17    if (sleep_time > NSEC_PER_MSEC)
18      p->wakeup_boost_pending = 1;
19  }
```

Listing 3: Wakeup qualification in kernel/sched/fair.c

```
1   if (flags & DEQUEUE_SLEEP) {
2     unsigned int state = READ_ONCE(p->__state);
3
4     p->wakeup_boost_pending = 0;
5     if (state & TASK_UNINTERRUPTIBLE)
6       p->last_uninterruptible_sleep_start = rq_clock(rq);
7     else
8       p->last_uninterruptible_sleep_start = 0;
9   }
```

Listing 4: Sleep timestamp in dequeue_task_fair

```
1   se->vruntime = vruntime - lag;
2
3   /* Apply one bounded boost to the qualifying wakeup. */
4   if ((flags & ENQUEUE_WAKEUP) && entity_is_task(se)) {
5     struct task_struct *tsk = task_of(se);
6
7     if (tsk->wakeup_boost_pending) {
8       u64 boost = sysctl_sched_base_slice;
9       u64 floor = cfs_rq->min_vruntime - vslice;
10
11      se->vruntime -= calc_delta_fair(boost, se);
12
13      if ((s64)(se->vruntime - floor) < 0)
14        se->vruntime = floor;
15      tsk->wakeup_boost_pending = 0;
16    }
17  }
```

Listing 5: Changes to place_entity in kernel/sched/fair.c

3.1.2 Wakeup Detection Logic

The logic uses `TASK_UNINTERRUPTIBLE` transitions as a practical heuristic for an I/O-like sleep. This state is also used by waits which are not real I/O, therefore the patch is detecting a scheduling pattern rather than the exact cause of the sleep.

- **Dequeue Hook:** When a task enters an uninterruptible sleep, `last_uninterruptible_sleep_start` captures the current `rq_clock`.
- **Enqueue Hook:** Upon wakeup, the sleep duration is calculated as $(T_{now} - T_{sleep})$. If this duration exceeds a **1ms threshold** $(1,000,000$ ns), the task is marked as eligible for one wakeup boost, and `wakeup_boost_pending` is set to 1.

These hooks are independent from the scheduler-statistics update paths. The benchmark still enables the `sched_schedstats` sysctl because `wait_sum`, the measured value, is a scheduler statistic.

3.1.3 Applying the Boost

In the fair scheduler, lowering `vruntime` can improve the task's eligibility and can also influence the virtual deadline calculated during placement. This is a temporary scheduling advantage rather than a change to the task's static priority. The modification manipulates `vruntime` in the `place_entity` function:

$$vruntime_{new} = vruntime_{original} - \text{calc\_delta\_fair}(\texttt{sysctl\_sched\_base\_slice}, task) \quad (1)$$

Subtracting one weight-adjusted standard base slice from the waking task's `vruntime` effectively "refunds" virtual time. Under EEVDF this can make the task eligible sooner and give it an earlier virtual deadline, but it does not guarantee immediate preemption in every scheduler state. To preserve a bound, the resulting `vruntime` is clamped so it does not drop below the runqueue's minimum `vruntime` minus one virtual slice. If EEVDF retained the sleeping entity through delayed dequeue, the patch removes it from the deadline-ordered tree before changing its placement and then inserts it again, keeping the tree ordering and augmented data consistent.

4 Methodology

4.1 Benchmarking Environment

To isolate scheduler mechanics from hardware jitter, the benchmark is conducted in a controlled QEMU environment:

- **Host:** Ubuntu 22.04, with the QEMU process restricted to isolated cores (`taskset -c 2,3`) and assigned real-time priority (`chrt -r 99`) to reduce host-side interference.
- **Guest:** QEMU emulation of an ARM Cortex-A9 (Vexpress), running Linux Kernel 6.13.9 with Full Preemption enabled (`CONFIG_PREEMPT=y`) and a 1000Hz Timer.
- **Virtualization:** `accel=tcg` with `thread=multi` allows QEMU to execute guest vCPUs on separate host threads. The benchmark does not use `-icount`, therefore this setup is not instruction-count deterministic; `-rtc clock=vm` only makes the guest clock follow virtual time.

This phase of project development was the most important and the one which was the most time consuming (around 2 months). It is really important to understand why; which is that when trying to verify if a modification is properly designed and applied to the kernel, it must be compared to a benchmark run with the vanilla kernel (unmodified). But, if the benchmarking environment is heavily affected by noise it is impossible to measure meaningful data. For each run of the vanilla and the patched kernel, each would record very different data from the previous runs; in the case of few iterations, the risk of getting fooled by randomness is very high (intended as taking future design decisions based on trash data). With a high number of iterations (let's say 100) part of the noise can be averaged out, but the run also lasts many hours, therefore reducing it before starting was still important.

To solve this, working on a PC running Linux natively (rather than in a virtual machine) is essential because GRUB can be configured to isolate one or more cores at boot time and reserve them for specific tasks. In this setup, the isolated cores were used through the QEMU configuration as follows:

- Modify this line in /etc/default/grub to isolate CPU cores 2 and 3:
  GRUB_CMDLINE_LINUX_DEFAULT="quiet splash isolcpus=2,3"
- Run `sudo update-grub` and reboot after changing the GRUB configuration.
- Restrict the complete QEMU process to the host CPU set composed of cores 2 and 3. This does not enforce a permanent one-to-one mapping between a host core and a guest vCPU.
- Inside the guest, pin both the CPU-bound workload and the I/O-like target to vCPU 1, thus effectively creating contention on the same guest runqueue. Guest vCPU 0 remains available for other operating-system work.

By doing so, host-side interference was reduced enough to obtain measurements which were much more repeatable than the first runs.

4.2 Workloads

Custom C executables generate controlled load scenarios. The number of processes is tunable from the main `wakeup_boost_test.sh` script:

- `cpu_load` **(Noise):** A math-heavy process calculating sine, cosine, and square roots in an infinite loop. Multiple instances (1 to 12) are spawned to saturate the runqueue.
- `io_load` **(Target):** A process that reads from a custom kernel module (`mychardev`). The module forces a 2ms uninterruptible sleep per read operation, creating a controlled synthetic I/O-like pattern.

To further reduce noise, the frequent printf calls used inside the read loop during the debug phase are removed since terminal output was one of the instructions which slowed execution the most.

4.3 Metric

The primary metric used is `wait_sum`, retrieved from the kernel's internal scheduler statistics via `/proc/[pid]/sched`. Unlike wall-clock time, which is dominated by the sleep duration itself, `wait_sum` specifically measures the time a task spent *runnable* (waiting on the runqueue) but *not running*. This focuses the measurement on the latency introduced while waiting for the scheduler. When at least four samples are available, the analyzer removes outliers via Interquartile Range before calculating the final averages.

5 Validation Status

The quantitative results from the previous implementation are intentionally not included because they do not validate the final patch documented here. The patch applies cleanly to Linux 6.13.9 and the modified scheduler objects compile, but a fresh paired QEMU run is still required before making a performance claim.

The practical test uses 6 competing CPU-bound processes and 3 main iterations. The complete performance run keeps the longer matrix of 1 through 12 competing processes and 100 iterations for each scenario. The benchmark first measures the unmodified fair scheduler, then applies the patch and repeats the same workload. Generated logs and graphs remain local under `.build/results/`.

6 Conclusion

The implementation of the one-shot wakeup boost is complete. Identifying a wakeup after a long uninterruptible sleep and applying a bounded `vruntime` bonus can improve EEVDF eligibility and virtual deadline placement without changing the task's static priority. It does not detect block I/O directly and does not guarantee immediate preemption.

There are still limits which are important to keep clear: the workload is synthetic, the unmodified run is completed before the patched run rather than interleaving their samples, and the experiment does not measure CPU throughput, starvation or wider system stability. The performance effect of the final implementation must be established by the new benchmark run before the project can report a quantitative improvement.